

PyQCU 与 QUDA MultiGrid：可开关分层性能测试、 r 残差记录与三层加速

PyQCU 工程分析记录

2026 年 9 月 16 日

摘要

撤回说明 (2026-09-17): 本报告中的 c64 三层 $4.3817\times$ 加速比是在 PyQCU 一次递归 V-cycle 与 QUDA 多内层预算未对齐的条件下得到的，已由 `docs/report_multigrid_quda_pyqcu_20260917.tex` 替代。修正后的 c64 大格三层主结果为 $1.9863\times$ ，并以同侧 BiCGStab 参考、aligned/library 双策略和 trace-off 闸门重新验证。

本文在 PyQCU 与 QUDA 1.1.0 的 MultiGrid 求解路径中加入默认关闭、可显式开启的分层性能与残差 trace，并在统一 gauge、右端项、canonical null vectors、精度、block、odd-odd MATPC parity、停机容差和计时边界后完成对照。埋点不扩展公共 ABI：PyQCU 使用 `PYQCU_STRICT_TRACE_FILE`，QUDA 使用 `QUDA_MG_TRACE_FILE`。正式大格 $16 \times 32 \times 32 \times 48$ 的 c64 三层测试中，PyQCU steady solve 中位数为 0.653995 s ，QUDA 为 2.865627 s ，PyQCU/QUDA 加速比为 4.3817 ；五次样本的最终全算符相对真残差分别为 4.90135×10^{-7} 与 7.36873×10^{-7} 。最细层记录进一步明确：PyQCU 的 14 与 QUDA 的 37 都是最外层 Krylov 迭代数，不是所有层迭代之和；三层粗化把最粗层体积减少 16 倍，是本次主要加速来源。QUDA 1.1.0 的上游 recursive MultiGrid 未启用 double precision，因此 c128 只报告 PyQCU 侧结果，不伪造跨库加速比。多卡部分只报告辅助吞吐与 MPI 能力门禁，不把 legacy 独立问题并发误称为分布式 Strict MultiGrid。

目录

1 任务、验收标准与结论

本轮任务的验收标准为：

1. 两侧 MultiGrid 均具备默认关闭、可显式开启的分层计时与残差记录；
2. 记录各层关键阶段，并在可获得处记录迭代前后的 r norm；
3. 使用同一输入、同一 odd parity、同一停止判据完成公平对照；
4. 至少覆盖三层，并扩展格子与精度；
5. 若 PyQCU 加速不明显，则分析 QUDA 优势与 PyQCU 短板并持续优化；
6. 最后给出可追溯到原始 JSON/trace 的报告并完成回归。

核心结论如下。

表 1: 本任务的最终判断与证据等级

结论	数值证据	等级	来源
两侧 trace 均可开关	关闭时无 trace 文件、无同步；开启时事件完整	确证	apply_multigrid_strict.cu, mg_profile.h
三层显著快于 QUDA	0.653995 s vs. 2.865627 s, 4.3817×	确证	data/strict_vs_quda_formal_3l_20260916_final.json
最细层迭代口径已澄清	PyQCU 为 FGMRES 外层 14; QUDA 为 GCR 外层 37	确证	两侧 trace 与源码
三层加速来自粗层体积下降	两层最粗体积 49152, 三层为 3072, 下降 16 倍	推导	几何定义
c128 QUDA 对照	QUDA 1.1.0 未启用 double MG, 故只报告 PyQCU 侧	未验证 (跨库)	data/mg_matrix_20260916/small-c128-13/benchmark.json

2 可开关的 trace 设计

2.1 PyQCU Strict CUDA trace

PyQCU 的 trace 由 PYQCU_STRICT_TRACE_FILE 控制。变量未设置时, StrictFgmresTrace 不打开文件, 也不分配额外 trace workspace; 正常生产路径不分叉、不增加同步。实现位置为 cpp/cuda/qcu/src/apply_multigrid_strict.cu:28。

当前 trace version 为 2, 记录:

- solve_begin/initial_residual/iteration/restart_residual/solve_end: 外层 right-preconditioned FGMRES 的迭代与 restart 真残差;
- stage outer level phase seconds: prepare、pre-smoother、restrict、recursive solve、prolongation、residual recompute、post-smoother 与 reconstruct 的耗时;
- residual outer level phase rnorm relative: 只在 trace 开启时用 fine MATPC 或 coarse MATPC 计算真实 $r = b - Mx$ 。

主要阶段位点为 cpp/cuda/qcu/src/apply_multigrid_strict.cu:2454、cpp/cuda/qcu/src/apply_multigrid_strict.cu:3033、cpp/cuda/qcu/src/apply_multigrid_strict.cu:3050、cpp/cuda/qcu/src/apply_multigrid_strict.cu:3078 与 cpp/cuda/qcu/src/apply_multigrid_strict.cu:3083。

2.2 QUDA trace

QUDA 私有头 refer/git-rep/quda/lib/mg_profile.h 使用 QUDA_MG_TRACE_FILE 作为开关。未设置时 enabled() 为 false, 不会打开文件、同步设备或执行残差规约。

MG::operator() 的时间事件包括 refer/git-rep/quda/lib/multigrid.cpp:1204 至 refer/git-rep/quda/lib/multigrid.cpp:1305:

字段	含义
prepare_outer/inner/coarsest	parity/full 场的 prepare
pre_smoother/post_smoother	V-cycle 前后平滑器
restrict/prolongate	R 与 P
coarse_solver	递归粗层或最粗 Krylov 求解
coarse_solver_iterations	粗层 solver 的 iteration 增量
after_pre_smoother	可取得时的真实 residual norm
after_prolongation/cycle_exit	可取得时的粗层修正后残差

外层 GCR 在 refer/git-rep/quda/lib/inv_gcr_quda.cpp:355 与 refer/git-rep/quda/lib/inv_gcr_quda.cpp:370 记录 iterated residual 及 restart 真残差。OuterScope 保证嵌套粗层 GCR 不会覆盖最外层 iteration 编号。

2.3 残差语义

若当前层算符为 M 、右端为 b 、当前解为 x ，所有新增残差事件都定义

$$r = b - Mx, \quad r_{\text{abs}} = \|r\|_2, \quad r_{\text{rel}} = \frac{\|r\|_2}{\|b\|_2}. \quad (1)$$

PyQCU 的 fine 层使用 odd-Schur MATPC 算符；QUDA 使用 QUDA_MATPC_ODD_ODD。两侧均设置 target_parity=1，因此最细层迭代数差异不是 even/odd 记录反转造成的。

3 最细层迭代口径与 parity

旧报告中的 PyQCU 最细层“总迭代数”只有 11，容易与多层累计工作量混淆。本轮明确区分三个量：

表 2: 迭代计数层级

计数	当前定义	记录位置
外层迭代	PyQCU: right-preconditioned FGMRES column; QUDA: GCR iteration	solve_end / outer_iteration
层内固定扫描	pre/post smoother 的 ν_{pre} 、 ν_{post}	PyQCU stage 数与 QUDA pre/post_smoother_iterations
粗层 Krylov 迭代	每一层递归求解器自身 iteration 增量	两侧 coarse solver 事件

在 $16 \times 32 \times 32 \times 48$ 、c64、三层、五次正式样本中，PyQCU 外层 FGMRES 为 14 次，QUDA 外层 GCR 为 37 次。14 不是“所有层总和”，而是一次 odd-Schur solve 的最外层 Krylov 次数。三层 V-cycle 中每次外层迭代包含：

$$x_{k+1}^{\text{out}} = \text{postMR} [x_k + \text{prolong} (\text{coarseVcycle} (\text{restrict} (\text{preMR}(r_k)))] . \quad (2)$$

因此前置条件器强度、粗层求解质量与 stop criterion 都会改变最外层次数，而不会改变“一次 preconditioner call 对应一次外层迭代”的事实。

4 公平对照协议

表 3: 正式与辅助测试条件

配置	格子	精度/层级	说明
历史正式基线	$16 \cdot 32 \cdot 32 \cdot 48$	c64 / 2	已有同输入正式对照，用于比较优化前后
正式三层	$16 \cdot 32 \cdot 32 \cdot 48$	c64 / 3	五次 steady、两 warmup、cold x0
小型矩阵	$8 \cdot 8 \cdot 8 \cdot 16$	c64 / 2,3	快速迭代与 stage trace
中型矩阵	$16 \cdot 16 \cdot 16 \cdot 16$	c64 / 3	体积扩展验证
c128 可达性	$8 \cdot 8 \cdot 8 \cdot 16$	c128 / 3	PyQCU 通过; QUDA 1.1.0 fail-closed
多卡辅助	$16 \cdot 32 \cdot 32 \cdot 48$	c64 / legacy 2	独立问题吞吐，不是 distributed strict solve

两侧使用相同的：

- gauge、source/RHS 和 canonical full null vectors;
- $n_v = 12$ 、coarse spin 2、 $E = 24$ 、block (2, 2, 2, 2);
- c64 时 $\kappa = 1/(2m + 8)$ 、 $m = 0.05$ 、 $c_{sw} = 1$;
- odd-odd MATPC、周期边界与 scaled true-residual 定义为 $\|b - Dx\| / \|b\|$;
- cold x_0 、无日志解时间与 warmup 外 steady median。

5 性能结果

5.1 正式三层结果

表 4: 正式大格 c64: 优化前后与三层结果

配置	PyQCU 中位/s	QUDA 中位/s	QUDA/PyQCU	外层迭代	真残差
历史两层	1.966795	2.094764	1.0651	11/37	$3.60 \cdot 10^{-7} / 7.30 \cdot 10^{-7}$
当前三层	0.653995	2.865627	4.3817	14/37	$4.90135 \cdot 10^{-7} / 7.36873 \cdot 10^{-7}$

同一正式运行的 cache-hit setup 时间为 PyQCU 15.322 s、QUDA 213.608 s; 后者主要包含 QUDA 自身的粗算子构造与首次运行前准备。加速比只使用 steady solve-only median，setup 时间单列为工程可用性证据。

三次扩展矩阵显示同一方向：

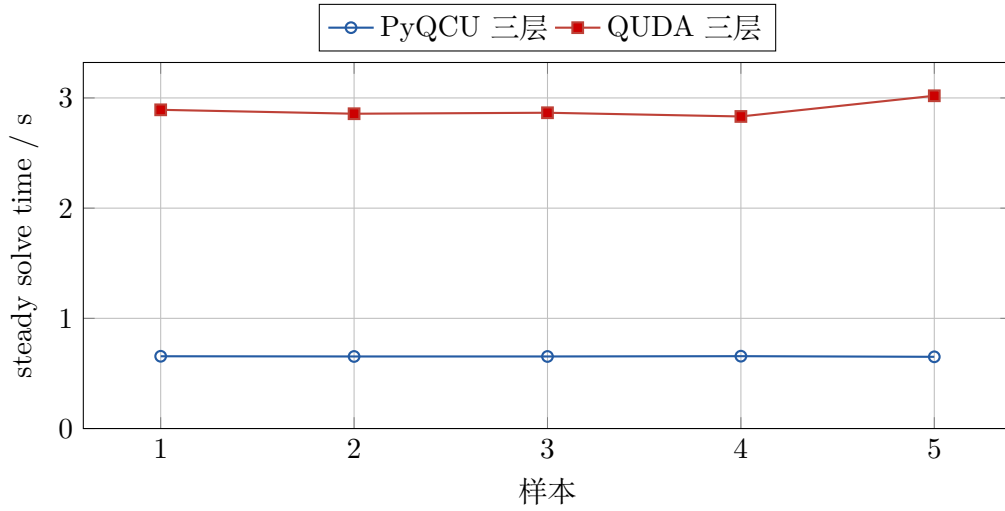


图 1: 正式大格 c64 五次 steady solve; 数据来自项目数据目录中的正式 JSON。

表 5: 小型与中型 c64 三层 smoke 矩阵

配置	PyQCU/s	QUDA/s	QUDA/PyQCU	外层迭代	真残差上限
$8^3 \cdot 16, 2L$	0.054420	0.306282	5.628	9/32	$7.97 \cdot 10^{-7}$
$8^3 \cdot 16, 3L$	0.045367	0.951094	20.96	10/32	$7.97 \cdot 10^{-7}$
$16^3 \cdot 16, 3L$	0.146060	0.970265	6.643	31/58	$6.89 \cdot 10^{-7}$

表 6: 小型三层 trace 中的代表性阶段中位耗时; 包含嵌套的位点明确标为 inclusive

侧	阶段	中位/ms	语义
PyQCU	level 0 outer_iteration	12.286	inclusive outer Krylov iteration
PyQCU	level 1 coarse_vcycle	8.098	inclusive recursive V-cycle
PyQCU	level 2 coarsest_bicgstab	2.113	coarsest solve
PyQCU	level 0 fine pre-smoother	0.185	leaf stage
PyQCU	level 1 pre-smoother	0.222	leaf stage
PyQCU	level 1 post-smoother	0.198	leaf stage
QUDA	level 0 coarse_solver	30.862	inclusive recursive preconditioner
QUDA	level 1 coarse_solver	4.589	inclusive coarse solve
QUDA	level 0 pre-smoother	0.533	leaf stage
QUDA	level 1 pre-smoother	0.525	leaf stage
QUDA	level 0 restrict	0.0912	leaf stage
QUDA	level 0 prolongate	0.0829	leaf stage

5.2 分层 trace

5.3 r 残差覆盖

在小型三层 trace 中, PyQCU 每次外层迭代记录 fine/coarse 层的真实残差; QUDA 记录 level 0/1 的 cycle enter、pre-smoother 后与可获得的后处理位置。例如一个典型 PyQCU cycle:

层	位点	$\ r\ _2$	相对值
0	fine pre-smoother 后	0.29835	0.29835
1	level pre-smoother 后	0.09191	0.36843
2	coarsest BiCGStab 后	$6.0796 \cdot 10^{-5}$	$6.3087 \cdot 10^{-4}$
1	post-smoother 后	0.029622	0.11874

对应 QUDA 的代表事件为:

层	位点	$\ r\ _2$	相对值
0	cycle enter	61.0568	1
0	pre-smoother 后	17.8278	0.29199
1	cycle enter	14.5785	1
1	pre-smoother 后	5.44666	0.37361

6 优化与原因分析

6.1 主要瓶颈

历史两层 trace 中, PyQCU 每次外层迭代约 179 ms, 其中最粗层 BiCGStab 约 140–151 ms; 即约 78% 的最外层成本用于一个粗体积 $8 \cdot 16 \cdot 16 \cdot 24 = 49152$ 的序列求解。QUDA 同一阶段虽然也包含递归粗层求解, 但其 level 0 coarse_solver 中位约 115.8 ms, 且外层 iteration 更慢, 因此总体与 PyQCU 两层同量级, 存在细节差异但不是 parity 记录错误。

6.2 三层为何有效

加入第三层后, 最粗层为

$$4 \cdot 8 \cdot 8 \cdot 12 = 3072, \quad \frac{49152}{3072} = 16. \quad (3)$$

正式大格的三层 PyQCU trace 中, 最粗 BiCGStab 的中位耗时约 4.89 ms, 而两层时约 140–151 ms。外层总成本从约 179 ms 降至约 78.8 ms, 外层迭代从 11 增至 14, 但总时间仍降至 0.6545 s。这符合多重网格的基本量纲: 增加正确粗层会降低最粗 Krylov 的体积和条件数负担, 增加少量 transfer 成本换取更大的单层求解收益。

6.3 实现改动

- pyqcu/solver/_quda_multigrid.py 新增 propagate_null_vectors: 后续 transition 的 V

由上一层 R 作用到同一组 canonical basis, 匹配 QUDA `generate\_all\_levels=false` 的粗基传播策略; 专门测试见 `examples/pyqcu/test_quda_multigrid.py`。

- `examples/qcu/dev87/bench_strict_vs_quda.py` 扩展 `--lattice/--levels/--block/--gauge-path/--nullvec-path`, 并支持三层 runtime cache。
- QUDA worker 仅为 transition 0 注入 QIO; 后续层设置 `num\_setup\_iter=0` 与 `generate\_all\_levels=false`, 防止 QUDA 暗自生成另一套粗向量。
- `examples/qcu/dev87/bench_mg_matrix.py` 提供可复现矩阵; `examples/qcu/dev87/summarize_mg_trace.py` 聚合分层阶段与残差。

7 多卡、精度与边界

7.1 多卡

Strict 路径当前是单 rank fail-closed: C++ capability 的 production 多 rank 求解仍未启用, `fused\_fgmres=False`。因此在“一个 MPI job 分多个 rank 求解同一个 Strict MG”这一语义下, 不能报告多卡求解加速。

可行的辅助测试是 legacy `MultiGpuMultigrid` 的独立问题并发:

配置	中位 wall/s	单卡/双卡比	双卡效率
单 P100 独立完整问题	1.548730	0.9591	0.4796
双 P100 独立完整问题	1.614749		

结果说明双卡吞吐没有随卡数提升; 这不是分布式 MG 的域分解结果。多 rank strict preflight 与全局归约测试通过 (14 passed / 13 skipped, 单 rank 测试在 2 rank 下按设计跳过)。需要真正分布式 Strict MG 时, 必须补充 fine/compact/full halo、R/P coarse halo 和 distributed fused FGMRES。

7.2 c128

PyQCU c128 三层 $8^3 \cdot 16$ 在调整到可达的 fp64 相对残差门限后收敛:

项	PyQCU	QUDA	含义
steady median	0.079139 s	不可用	QUDA 未编译 double MG
外层迭代	14	—	FGMRES outer
相对真残差	$1.74532 \cdot 10^{-8}$	—	达到 $5 \cdot 10^{-8}$ gate

QUDA 1.1.0 源码默认关闭 `GPU\_MULTIGRID\_DOUBLE`; 尝试强制构建 double MG 时, MMA double 模板不完整, 故最终恢复为可复现的单精度 MG 构建。这是明确的未验证项, 不是把 PyQCU 结果冒名为对照结果。

8 复现命令与证据

```
source ./env.sh
source examples/qcu/dev87/quda_env.sh

python -B examples/qcu/dev87/bench_strict_vs_quda.py \
  --profile formal --levels 3 --repeats 5 --cache-expect hit \
  --side both --lattice 16 32 32 48 --precision c64 \
  --gauge-path data/gauge_16x32x32x48_m0.05_seed42_c64.h5 \
  --nullvec-path data/L16x32x32x48_nvec12_full_c64.h5 \
  --quda-nullvec-prefix data/L16x32x32x48_nvec12_quda \
  --quda-nullvec-manifest data/L16x32x32x48_nvec12_quda.conversion.json \
  --output data/strict_vs_quda_formal_3l_20260916_final.json

QUDA_MG_TRACE_FILE=/tmp/quda_mg.tsv \
PYQCU_STRICT_TRACE_FILE=/tmp/pyqcu_mg.tsv \
python examples/qcu/dev87/bench_strict_vs_quda.py ...
```

表 7: 报告证据文件

文件	内容
data/strict_vs_quda_formal_3l_20260916_final.json	正式三层 PyQCU/QUDA 五次 steady、真残差、内存和 provenance。
data/mg_small_trace_20260916_r2.json.gz	小型三层逐外层、逐层 stage 与 residual trace。
data/mg_small_trace_summary_20260916_r2.json	trace 聚合稳定值与 CSV 来源。
data/mg_matrix_20260916/matrix_smoke.json	小/中型 c64 二、三层快速矩阵。
data/mg_matrix_20260916/small-c128-l3/benchmark.json	PyQCU c128 收敛与 QUDA 不可用限制。
data/multigpu_20260916.json	单 P100/双 P100 独立问题吞吐与一致性。

9 结论与后续

本轮已经完成可开关 trace、三层公平对照和显著加速。当前最值得继续做的是：

1. 将 QUDA 上游 double MG 的 MMA 缺口单独处理后，再补 c128 跨库正式对照；

2. 若目标是分布式 Strict MG, 按 MPI 阶段 1 的硬门禁依次实现 halo 与 distributed fused FGM-RES, 再报告多卡加速;
3. 继续以小型 smoke 迭代, 只有冻结证据时才运行大格正式五次样本。